

Tecnologias e conceitos utilizados nas correções

Ferramenta/Conceito	Finalidade
JWT (jsonwebtoken)	Geração e verificação de tokens, com expiração (expiresIn) e segredo forte.
bcryptjs	Hash de senhas (já usado, mas manteremos).
Refresh Token (armazenado em banco ou Redis)	Permitir renovação de tokens sem expor credenciais novamente.
Middleware de ownership	Verificar se o objeto (produto) pertence ao usuário autenticado antes de qualquer operação.
Variáveis de ambiente (.env)	Armazenar segredos de forma segura, fora do código.
Remoção de endpoints sombra	Eliminar rotas não documentadas ou protegê-las com autenticação e autorização adequadas.
Logging e monitoramento	Detectar acessos indevidos (não implementado em detalhes, mas citado como boa prática).

1. Correção da Broken Authentication (auth-service)

Problemas originais

- JWT sem expiração (**expiresIn** não definido).
- Segredo do JWT fraco e previsível.
- Nenhum mecanismo de refresh token.
- Senhas armazenadas com bcrypt (ok), mas sem logout ou 2FA (não será corrigido aqui por simplicidade).

Correções aplicadas

1. **Segredo forte** – usar variável de ambiente com valor complexo (ex.: **JWT_SECRET=7f3e8a9c...**).
2. **Expiração curta** – definir **expiresIn: '15m'** (15 minutos).
3. **Refresh token** – endpoint **/refresh** que recebe um refresh token válido (armazenado no banco) e emite um novo access token.
4. **Refresh token rotativo** – cada uso gera um novo refresh token, invalidando o anterior.

Código corrigido (auth-service/routes/auth.js)

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const db = require('../db');
const router = express.Router();
require('dotenv').config();
```

```
const JWT_SECRET = process.env.JWT_SECRET; // agora obrigatório e forte
const REFRESH_SECRET = process.env.REFRESH_SECRET || JWT_SECRET; // idealmente
outro segredo

// Gerar access token (curta duração)
function generateAccessToken(user) {
  return jwt.sign(
    { id: user.id, username: user.username },
    JWT_SECRET,
    { expiresIn: '15m' } // ✅ expira em 15 minutos
  );
}

// Gerar refresh token (longa duração, armazenado)
function generateRefreshToken(user) {
  const refreshToken = jwt.sign(
    { id: user.id, type: 'refresh' },
    REFRESH_SECRET,
    { expiresIn: '7d' }
  );
  return refreshToken;
}

// Registro (sem alteração)
router.post('/register', async (req, res) => {
  const { username, password } = req.body;
  if (!username || !password) return res.status(400).json({ error: '...' });
  try {
    const hashed = await bcrypt.hash(password, 10);
    const [result] = await db.execute('INSERT INTO users (username, password)
VALUES (?, ?)', [username, hashed]);
    res.status(201).json({ id: result.insertId, username });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Login - retorna access + refresh
router.post('/login', async (req, res) => {
  const { username, password } = req.body;
  try {
    const [rows] = await db.execute('SELECT * FROM users WHERE username = ?',
[username]);
    if (rows.length === 0) return res.status(401).json({ error: 'Credenciais
inválidas' });
    const user = rows[0];
    const valid = await bcrypt.compare(password, user.password);
    if (!valid) return res.status(401).json({ error: 'Credenciais inválidas' });

    const accessToken = generateAccessToken(user);
    const refreshToken = generateRefreshToken(user);

    // Armazena refresh token no banco (associado ao usuário)
    await db.execute('UPDATE users SET refresh_token = ? WHERE id = ?',
```

```
[refreshToken, user.id]);

    res.json({ accessToken, refreshToken });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Renovação de access token
router.post('/refresh', async (req, res) => {
  const { refreshToken } = req.body;
  if (!refreshToken) return res.status(401).json({ error: 'Refresh token obrigatório' });

  try {
    const decoded = jwt.verify(refreshToken, REFRESH_SECRET);
    const [rows] = await db.execute('SELECT * FROM users WHERE id = ? AND refresh_token = ?', [decoded.id, refreshToken]);
    if (rows.length === 0) return res.status(403).json({ error: 'Refresh token inválido' });

    const user = rows[0];
    const newAccessToken = generateAccessToken(user);
    const newRefreshToken = generateRefreshToken(user);

    // Rotaciona refresh token
    await db.execute('UPDATE users SET refresh_token = ? WHERE id = ?', [newRefreshToken, user.id]);

    res.json({ accessToken: newAccessToken, refreshToken: newRefreshToken });
  } catch (err) {
    res.status(403).json({ error: 'Refresh token expirado ou inválido' });
  }
});

// Middleware de verificação (agora verifica expiração)
router.post('/verify', async (req, res) => {
  const token = req.body.token;
  if (!token) return res.status(401).json({ valid: false });
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    res.json({ valid: true, user: { id: decoded.id, username: decoded.username } });
  } catch (err) {
    res.status(401).json({ valid: false, error: err.message });
  }
});

module.exports = router;
```

O que foi corrigido:

- Access token expira em 15 minutos.

- Refresh token armazenado no banco, rotativo.
- Segredo agora é variável de ambiente obrigatória (não há fallback fraco).
- O middleware de verificação agora respeita `exp` e retorna erro se expirado.

2. Correção da BOLA (product-service)

Problemas originais

- Qualquer usuário autenticado pode listar, criar, alterar ou deletar produtos de qualquer outro usuário.
- O `userId` é aceito do corpo da requisição, sem validação com o token.

Correções aplicadas

1. **Remover `userId` do corpo da requisição** – o ID do usuário deve vir exclusivamente do token JWT.
2. **Adicionar filtro de ownership** em todas as consultas: `WHERE user_id = ?` com o valor extraído do token.
3. **Middleware próprio** para verificar propriedade antes de operações.

Código corrigido (product-service/routes/products.js)

```
const express = require('express');
const axios = require('axios');
const db = require('../db');
const router = express.Router();
require('dotenv').config();

// Middleware de autenticação (igual antes, mas agora retorna user)
const verifyToken = async (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader) return res.status(401).json({ error: 'Token não fornecido' });
  const token = authHeader.split(' ')[1];
  try {
    const response = await
    axios.post(`${process.env.AUTH_SERVICE_URL}/auth/verify`, { token });
    if (response.data.valid) {
      req.user = response.data.user; // { id, username }
      next();
    } else {
      res.status(401).json({ error: 'Token inválido' });
    }
  } catch (err) {
    res.status(401).json({ error: 'Falha na verificação' });
  }
};

//  Listar apenas produtos do usuário logado
router.get('/', verifyToken, async (req, res) => {
  try {
    const [rows] = await db.execute('SELECT * FROM products WHERE user_id = ?',
    [req.user.id]);
```

```
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ✅ Buscar produto por ID - com verificação de ownership
router.get('/:id', verifyToken, async (req, res) => {
  const { id } = req.params;
  try {
    const [rows] = await db.execute('SELECT * FROM products WHERE id = ? AND user_id = ?', [id, req.user.id]);
    if (rows.length === 0) return res.status(404).json({ error: 'Produto não encontrado' });
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ✅ Criar produto - userId vem do token, não do body
router.post('/', verifyToken, async (req, res) => {
  const { name, price } = req.body;
  if (!name || !price) {
    return res.status(400).json({ error: 'name e price são obrigatórios' });
  }
  try {
    const [result] = await db.execute(
      'INSERT INTO products (name, price, user_id) VALUES (?, ?, ?)',
      [name, price, req.user.id] // ✅ userId do token
    );
    res.status(201).json({ id: result.insertId, name, price, userId: req.user.id });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ✅ Atualizar - verifica ownership
router.put('/:id', verifyToken, async (req, res) => {
  const { id } = req.params;
  const { name, price } = req.body;
  try {
    const [result] = await db.execute(
      'UPDATE products SET name = ?, price = ? WHERE id = ? AND user_id = ?',
      [name, price, id, req.user.id]
    );
    if (result.affectedRows === 0) return res.status(404).json({ error: 'Produto não encontrado ou não pertence ao usuário' });
    res.json({ message: 'Produto atualizado' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
// ✅ Deletar - verifica ownership
router.delete('/:id', verifyToken, async (req, res) => {
  const { id } = req.params;
  try {
    const [result] = await db.execute('DELETE FROM products WHERE id = ? AND
user_id = ?', [id, req.user.id]);
    if (result.affectedRows === 0) return res.status(404).json({ error: 'Produto
não encontrado ou não pertence ao usuário' });
    res.status(204).send();
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ⚠️ Improper Inventory Management - veremos na seção 3
```

O que foi corrigido:

- `userId` não é mais aceito pelo corpo; apenas o token define o proprietário.
- Todas as queries incluem `AND user_id = ?`.
- Retorno 404 genérico não revela se o produto existe ou não (boa prática).

🔧 3. Correção do Improper Inventory Management

Problema original

- Endpoint `/internal/debug` exposto, sem autenticação, documentação ou controle.

Correções aplicadas

1. **Remover o endpoint** se não for mais necessário.
2. **Ou protegê-lo** com autenticação forte e restringir a administradores.
3. **Adicionar logging** para monitorar acessos (não implementado, mas recomendado).

Código corrigido (product-service/routes/products.js)

```
// ❌ REMOVER completamente o bloco abaixo:
/*
router.get('/internal/debug', async (req, res) => {
  // ... código perigoso ...
});
*/

// ✅ Se for realmente necessário um endpoint de debug, criar um endpoint
separado,
// protegido com autenticação e autorização (role admin).
// Exemplo (adicione ao routes/products.js):

const checkAdmin = (req, res, next) => {
```

```
// Supondo que o token contenha um campo 'role'
if (req.user.role !== 'admin') return res.status(403).json({ error: 'Acesso
negado' });
next();
};

router.get('/admin/debug', verifyToken, checkAdmin, async (req, res) => {
  const [products] = await db.execute('SELECT id, name, price, user_id FROM
products');
  res.json({
    message: 'Debug autorizado',
    total: products.length,
    products
  });
});
```

O que foi corrigido:

- Endpoint sombra removido.
- Se houver necessidade legítima, o novo endpoint `/admin/debug` exige autenticação (`verifyToken`) e autorização (`checkAdmin`).
- Documentação deve ser adicionada (Swagger/OpenAPI) para que não seja um serviço esquecido.



Boas práticas adicionais (não implementadas no código, mas recomendadas)

Prática	Descrição
Rate limiting	Limitar tentativas de login e refresh para evitar brute force.
Revogação de tokens	Manter blacklist de tokens ou usar refresh token rotativo com invalidação.
HTTPS	Todas as comunicações devem ser criptografadas em produção.
Logs de segurança	Registrar tentativas de acesso indevido (ex.: token inválido, ownership violada).
Testes automatizados	Incluir testes que verifiquem ownership (ex.: usuário A não pode acessar recurso de B).



Resumo das correções

Vulnerabilidade	Correção principal	Tecnologias
Broken Authentication	JWT com expiração, refresh token rotativo, segredo forte	<code>jsonwebtoken</code> , banco de dados para armazenar refresh tokens
BOLA	Remover <code>userId</code> do corpo; filtrar todas as queries pelo <code>user_id</code> extraído do token	Middleware, SQL parametrizado

Vulnerabilidade	Correção principal	Tecnologias
Improper Inventory Management	Remover endpoint não documentado ou protegê-lo com autenticação e autorização	Controle de acesso baseado em roles